

**VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY**



**523H0145 – NGUYEN NHUT KHANG
523H0164 – NGUYEN TRAN HOANG NHAN
523H0117 – MAI HOANG THAI**

MIDTERM ESSAY

INTRODUCTION TO DIGITAL IMAGE PROCESSING

HO CHI MINH CITY, 2025

**VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY**



**523H0145 – NGUYEN NHUT KHANG
523H0164 – NGUYEN TRAN HOANG NHAN
523H0117 – MAI HOANG THAI**

MIDTERM ESSAY

INTRODUCTION TO DIGITAL IMAGE PROCESSING

Advised by
Dr. Nguyen Chi Thien

HO CHI MINH CITY, 2025

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Dr. Nguyen Chi Thien, our instructor and mentor, for his valuable guidance and support throughout the mid-term report of our project. He has been very helpful and patient in providing us with constructive feedback and suggestions to improve our work. He has also encouraged us to explore new technologies and techniques to enhance our system's functionality and performance. We have learned a lot from his expertise and. We are honored and privileged to have him as our teacher and supervisor.

Ho Chi Minh city, 1st November 2025.

Author

(Signature and full name)

Khang

Nguyen Nhut Khang

Nhan

Nguyen Tran Hoang Nhan

Thai

Mai Hoang Thai

DECLARATION OF AUTHORSHIP

We hereby declare that this is our own project and is guided by Dr. Nguyen Chi Thien; The content research and results contained herein are central and have not been published in any form before. The data in the tables for analysis, comments and evaluation are collected by the main author from different sources, which are clearly stated in the reference section.

In addition, the project also uses some comments, assessments as well as data of other authors, other organizations with citations and annotated sources.

If something wrong happens, we will take full responsibility for the content of my project. Ton Duc Thang University is not related to the infringing rights, the copyrights that We give during the implementation process (if any).

Ho Chi Minh city, 1st November 2025

Author

(Signature and full name)

Khang

Nguyen Nhut Khang

Nhan

Nguyen Tran Hoang Nhan

Thai

Mai Hoang Thai

TABLE OF CONTENT

LIST OF FIGURES	vi
CHAPTER 1. Methodology of Solving Tasks.....	1
1.1 Task 1	1
<i>1.1.1 Short description of main programming steps.....</i>	<i>1</i>
<i>1.1.2 Detailed Source Code Explanation.....</i>	<i>2</i>
<i>1.1.3 Appendix for task 1</i>	<i>24</i>
1.2 Task 2	26
<i>1.2.1 Short description of main programming steps.....</i>	<i>26</i>
<i>1.2.2 Detailed Source Code Explanation.....</i>	<i>27</i>
CHAPTER 2. Task results	41
2.1 Task 1	41
<i>2.1.1 Output Image for Task 1</i>	<i>41</i>
<i>2.1.2 Output Video for Task 1</i>	<i>42</i>
2.2 Output Task 2	42

LIST OF FIGURES

Figure 1: Overall Pipeline Architecture diagram.....	1
Figure 2: Traffic Detection Pipeline per Frame	3
Figure 3: Preprocess Function.....	3
Figure 4: Preprocessing Pipeline.....	4
Figure 5: HSV boundaries.....	7
Figure 6: Morphological operations.....	8
Figure 7: Circle Detection Pipeline.....	10
Figure 8: Triangle Detection Pipeline	10
Figure 9: Circle Detection Function.....	11
Figure 10: Triangle Detection Function.....	12
Figure 11: Two-Layer ROI Filtering diagram	14
Figure 12: Two-Layer ROI Logic for circles.....	15
Figure 13: Two-Layer ROI Logic for triangles	15
Figure 14: is_bbox_in_roi function.....	15
Figure 15: Temporal Filter Class	17
Figure 16: Intersection over Union	17
Figure 17: Temporal Tracking Pipeline	18
Figure 18: add_detections function	19
Figure 19: Track Validation Pipeline.....	20
Figure 20: build_detection_cache function	21
Figure 21: Interpolation and Smoothing Pipeline	22
Figure 22: Linear Interpolation	22

Figure 23: interpolate_missing_frames function.....	23
Figure 24: smooth_bounding_boxes function.....	24
Figure 25: Multi-Processing and Multi-Threading diagram.....	25
Figure 26: Overall Pipeline Architecture diagram.....	26
Figure 27: Input Image after Dividing into 4 regions	28
Figure 28: Region 1 Pipeline Process diagram	28
Figure 29: Logical code for Part 1	28
Figure 30: Part 1 ROI with Bounding Boxes	29
Figure 31: Noise Region Pipeline Process diagram.....	30
Figure 32: Logical code for Noise Part.....	30
Figure 33: Binarization Noise Part ROI.....	31
Figure 34: Processed Noise Part ROI	32
Figure 35: Noise Part ROI with Bounding Box.....	33
Figure 36: Region 2 Pipeline Process diagram	33
Figure 37: Logical code for Part 2	34
Figure 38: Part 2 ROI with separated digit “4” in row 2	34
Figure 39: Part 2 ROI with digits “4” connect horizontal lines.....	35
Figure 40: Part 2 ROI with wrong Bounding Box of digit “24”.....	36
Figure 41: Part 2 ROI with contours_part_2 and bbox_for_part_2.....	36
Figure 42: Region 3 Pipeline Process diagram	37
Figure 43: Logical code for Part 3	37
Figure 44: The Processing Pipeline Image of Part 3 ROI.....	38
Figure 45: Recombination and Final Filtering Process Flowchart diagram	39

Figure 46: Offsets and Area Conditions	40
Figure 47: Applying Contours for Input Image	40
Figure 48: Output Frame 1	41
Figure 49: Output Frame 2	41
Figure 50: Output Frame 3	42
Figure 51 Final Result of task 2	43

CHAPTER 1. Methodology of Solving Tasks

1.1 Task 1

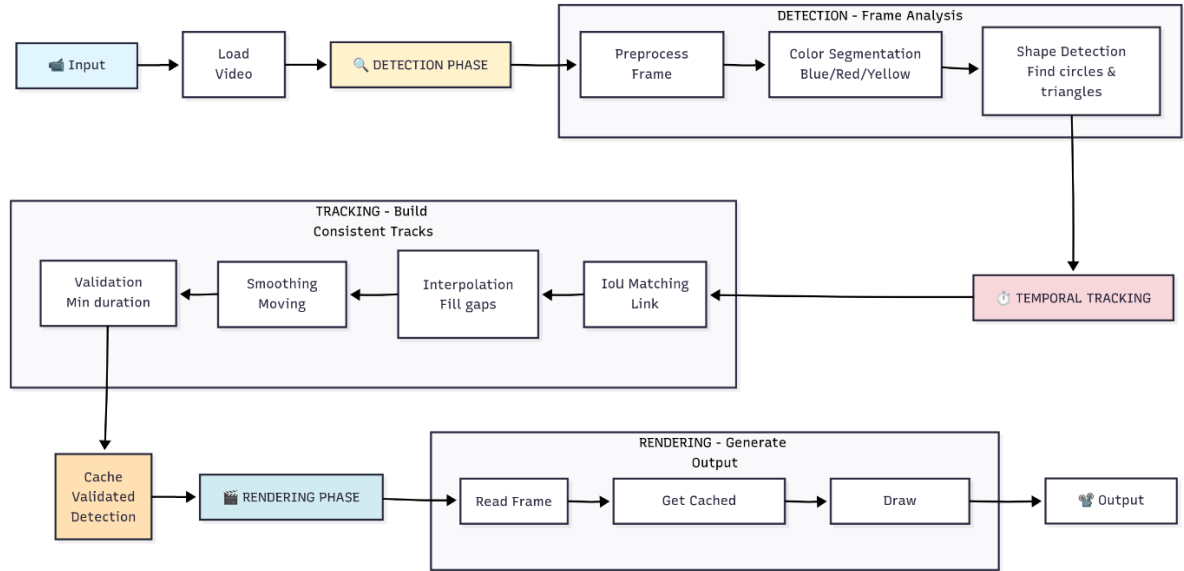


Figure 1: Overall Pipeline Architecture diagram

1.1.1 Short description of main programming steps

1. Detection Phase

This phase analyzes video frames up to a set limit to identify potential signs.

- **Image Preprocessing:** Frames are blurred, converted to HSV, and enhanced using CLAHE (Contrast Limited Adaptive Histogram Equalization) and Saturation Boost for robust color separation.
- **Color Segmentation:** Binary masks are created for Blue, Red, and Yellow using specific HSV ranges, followed by morphological cleaning to reduce noise and fill holes.
- **Shape Detection:** Contours are extracted from the clean masks and filtered based on Area, a Two-Layer ROI Logic (Region of Interest), and specific Shape Metrics.

2. Temporal Tracking

This phase validates the raw detections from the previous step and prepares stable tracks.

- **IoU Matching:** Detections are linked across frames using IoU (Intersection over Union) and a maximum time gap to build continuous tracks.
- **Validation:** Tracks that fail to meet the color-specific minimum duration are discarded as temporary noise.
- **Interpolation and Smoothing:** Valid tracks are refined using linear interpolation to fill small frame gaps and a moving average filter to smooth bounding box jitter.
- **Caching:** All validated, smoothed detections are stored in a cache for fast retrieval during the rendering phase.

3. Rendering Phase

This final phase uses a high-throughput, multi-threaded structure to generate the output video.

- **Frame Processing:** Reader threads feed frames to processor threads. The processor threads retrieve cached detections, draw the final bounding boxes, and place the annotated frame into an output queue.
- **Video Writing:** The main thread writes the processed frames in sequential order to the final video file.

1.1.2 Detailed Source Code Explanation

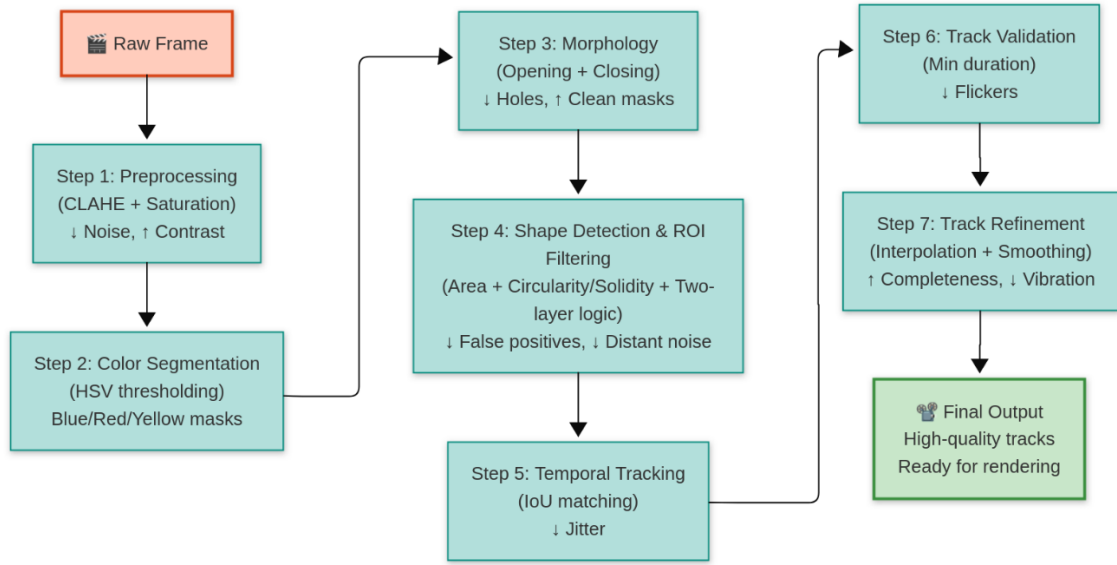


Figure 2: Traffic Detection Pipeline per Frame

1.1.2.1 Step 1: Preprocessing

The input frame is first converted from BGR to the HSV color space. To enhance contrast and illumination robustness, CLAHE is applied to the V (Value) channel. Additionally, the S (Saturation) channel is intentionally boosted to maximize color separation. Finally, Median Blur is applied to the frame to reduce noise before color analysis.

```

def preprocess_frame(frame, clip_limit, blur_ksize):
    """Preprocess frame with median blur, HSV conversion, CLAHE, and saturation boost"""
    blur_ksize = max(3, blur_ksize)
    if blur_ksize % 2 == 0:
        blur_ksize += 1

    frame_processing = cv2.medianBlur(frame, blur_ksize)
    frame_processing = cv2.cvtColor(frame_processing, cv2.COLOR_BGR2HSV)

    h, s, v = cv2.split(frame_processing)

    clahe = cv2.createCLAHE(clipLimit=clip_limit / CLAHE_CLIP_DIVISOR,
                             tileGridSize=CLAHE_TILE_GRID_SIZE)
    v_clahe = clahe.apply(v)

    # Boost saturation to enhance color separation
    s = s.astype(np.float32) * SATURATION_BOOST_FACTOR
    s = np.clip(s, 0, 255).astype(np.uint8)

    hsv_blur_clahe = cv2.merge([h, s, v_clahe])
    return hsv_blur_clahe
  
```

Figure 3: Preprocess Function

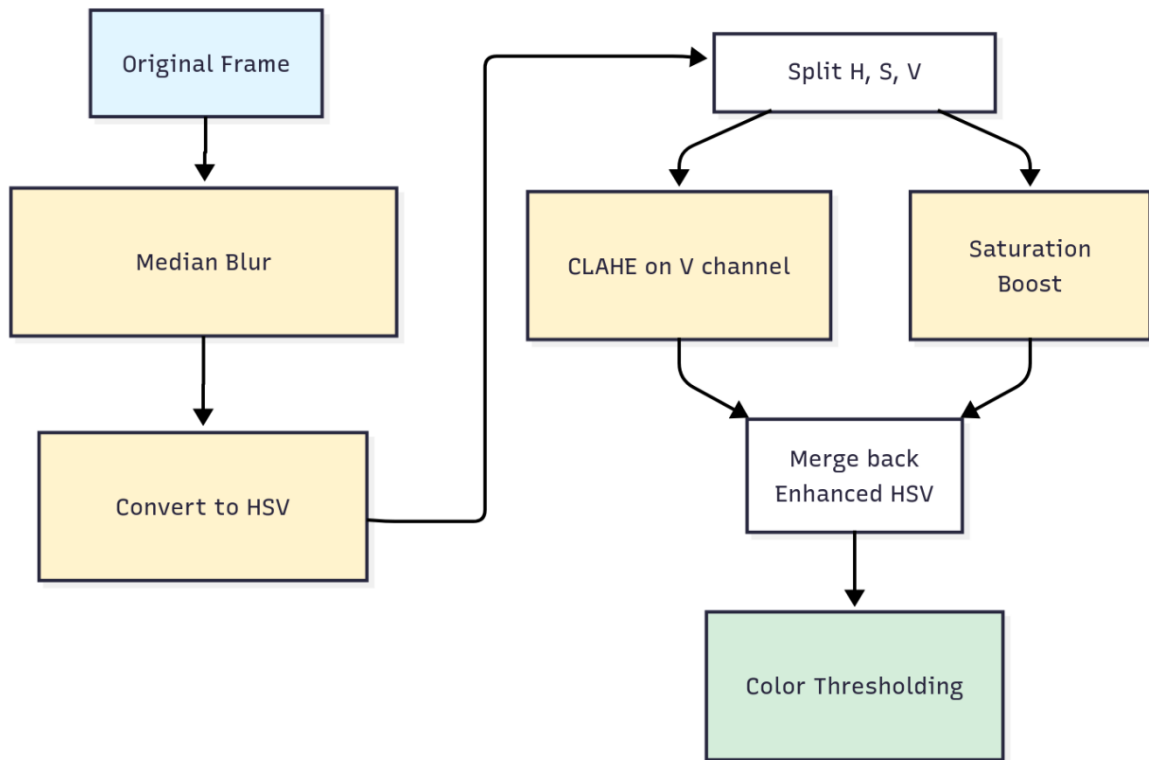


Figure 4: Preprocessing Pipeline

Median Blur Parameter Setup:

This section prepares the kernel size `blur_ksize` for the median blur filter:

- `blur_ksize = max(3, blur_ksize)`: It ensures the minimum kernel size is 3 to guarantee at least some degree of initial noise reduction.
- The if condition ensures `blur_ksize` is always an odd number, which is a requirement for the `cv2.medianBlur` function to define a central pixel.

Noise Reduction and Color Space Conversion:

Median Blur: `cv2.medianBlur` applies the blur using the prepared `blur_ksize`. The median filter is effective at reducing impulsive noise (like salt-and-pepper noise) while preserving image edges better than Gaussian blur.

BGR to HSV Conversion:

- The HSV (Hue–Saturation–Value) color model was chosen instead of RGB/BGR because it separates color information (Hue) from

brightness (Value). This makes the detection more robust under varying lighting conditions or shadows.

- In RGB, small illumination changes can drastically alter pixel values, whereas in HSV, hue remains relatively stable. As a result, thresholding on hue and saturation produces more consistent color segmentation for traffic signs captured under sunlight, shade, or reflection.

Split Channels and CLAHE Application:

Split Channels: The HSV frame is split into its three component channels: Hue, Saturation, and Value.

CLAHE Setup: CLAHE is created to enhance the V channel. This technique improves the contrast locally across the image, which helps make the signs stand out from the background, especially in poor lighting.

- `CLAHE_CLIP_DIVISOR = 10.0`: This global variable divides the color-specific `clip_limit` parameter to normalize the effective clip limit for CLAHE. Normalizing the clip limit prevents over-amplification of noise in homogeneous areas.
- `CLAHE_TILE_GRID_SIZE = (1, 1)`: This is the size of the grid for histogram equalization. A size of (1, 1) means a global histogram is used, essentially performing non-adaptive Contrast Limited Histogram Equalization (CLHE) on the entire image, which is faster and avoids tile boundary artifacts.

Saturation Boosting:

This step enhances the S channel to make the traffic sign colors more vibrant and distinct, improving the reliability of the color thresholding step.

- The S channel array is converted to float32 to allow for multiplication.

- `SATURATION_BOOST_FACTOR = 1.5`: This global variable multiplies the saturation values by 1.5. A mild boost is chosen to make colors pop without distorting the image too much.
- **Clipping and Conversion**: The boosted values are clipped between 0 and 255 and then converted back to `uint8` to match the original image data type.

Merge and Return:

The three modified channels - the original Hue, the boosted Saturation, and the contrast-enhanced Value are merged back together using `cv2.merge` to form the final preprocessed HSV image, ready for color thresholding.

1.1.2.2 Step 2: Color Segmentation

Using the preprocessed HSV frame, color-specific binary masks are created for Blue, Red, and Yellow using the `cv2.inRange` function. These isolates pixels falling within the HSV threshold ranges defined by global variables. This step results in three separate binary masks, one for each relevant traffic sign color.

Independent Color Pipelines:

- Red, blue, and yellow signs are processed separately, not through a combined mask.
- Each color requires distinct HSV thresholds, morphological kernels, and ROI positions.
- Combining them would cause interference — for instance, red noise could affect the yellow detection mask.
- Independent processing ensures color-specific tuning and accurate temporal tracking per category.

```

# Process BLUE
hsv_blue = preprocess_frame(frame_to_process, blue_clahe_clip_limit, blue_blur_ksize)
lower_blue = np.array([blue_lower_h, blue_lower_s, blue_lower_v])
upper_blue = np.array([blue_upper_h, blue_upper_s, blue_upper_v])
mask_blue = cv2.inRange(hsv_blue, lower_blue, upper_blue)
mask_blue_clean = morphology(mask_blue, blue_ksize, blue_open_iter, blue_close_iter)

# Process RED
hsv_red = preprocess_frame(frame_to_process, red_clahe_clip_limit, red_blur_ksize)
lower_red = np.array([red_lower_h, red_lower_s, red_lower_v])
upper_red = np.array([red_upper_h, red_upper_s, red_upper_v])
mask_red = cv2.inRange(hsv_red, lower_red, upper_red)
mask_red_clean = morphology(mask_red, red_ksize, red_open_iter, red_close_iter)

# Process YELLOW
hsv_yellow = preprocess_frame(frame_to_process, yellow_clahe_clip_limit, yellow_blur_ksize)
lower_yellow = np.array([yellow_lower_h, yellow_lower_s, yellow_lower_v])
upper_yellow = np.array([yellow_upper_h, yellow_upper_s, yellow_upper_v])
mask_yellow = cv2.inRange(hsv_yellow, lower_yellow, upper_yellow)
mask_yellow_clean = morphology(mask_yellow, yellow_ksize, yellow_open_iter, yellow_close_iter)

```

Figure 5: HSV boundaries

Preprocessing Input:

The process begins by receiving the frame that has already undergone preprocessing (noise reduction, BGR-to-HSV conversion, and contrast/saturation enhancement). This ensures the color information is distinct and robust to lighting variations.

Defining Color Ranges:

For each target color (Blue, Red, and Yellow), the code uses global parameters to define a specific range of Hue, Saturation, and Value (HSV). These parameters represent the optimal color boundaries for traffic signs in the video.

Creating Binary Masks:

The core operation uses a function to perform in-range thresholding. This function compares every pixel in the preprocessed HSV frame against the defined color range:

- If a pixel's HSV values fall within the specified upper and lower bounds, it is set to white (foreground).
- If a pixel's HSV values fall outside the range, it is set to black (background).

- This results in three separate binaries where only the potential traffic sign regions are visible as white blobs.

Transition to Morphological Cleaning:

Immediately following the creation of the raw binary mask, the result is passed directly to the next step, Morphological Cleaning (Step 3), to refine the mask and prepare it for contour detection.

1.1.2.3 Step 3: Morphological cleaning

The raw binary masks are refined using a custom morphology utility function. This involves applying a sequence of Opening (Erosion followed by dilation) to remove small noise specks, and Closing (Dilation followed by erosion) to fill minor holes within the sign candidates. This process generates clean, consolidated masks ready for contour extraction.

```
def morphology(mask, k_size, iter_opening, iter_close):
    """Apply morphological operations"""
    k_size = max(1, k_size)
    if k_size % 2 == 0:
        k_size += 1
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, ksize=(k_size, k_size))
    mask_clean = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel, iterations=iter_opening)
    mask_clean = cv2.morphologyEx(mask_clean, cv2.MORPH_CLOSE, kernel, iterations=iter_close)
    return mask_clean
```

Figure 6: Morphological operations

Kernel Preparation:

The kernel size is forced to be odd (if $k_size \% 2 == 0 \rightarrow +1$) so that it has a clear pixel, ensuring symmetric morphological behavior around each pixel.

Operation:

Morphological opening (erosion to dilation) removes small noise, while closing (dilation to erosion) fills small gaps in the detected mask, so opening was used before closing.

Opening Operation (Noise Removal):

- The first operation applied is Opening, which is performed a specified number of times.

- Action: Opening involves Erosion followed by a Dilation.
- Purpose: It effectively removes small, isolated specks of noise that might have been picked up during color segmentation, clearing debris from the mask.

Closing Operation (Hole Filling):

- The second operation is Closing, also applied a specified number of times.
 - Action: Closing involves a Dilation followed by an Erosion.
 - Purpose: It works to fill small gaps and holes within the foreground objects, consolidating the shape of the potential traffic sign and preventing fragmentation.

The result is a clean, simplified, and consolidated binary mask that is much better suited for the precise boundary and shape analysis required in the subsequent filtering steps.

1.1.2.4 Step 4: Shape Detection and ROI filtering

The Shape and ROI Filtering step takes the clean binary masks and identifies valid traffic signs by applying a multi-layered filter based on Area, Shape Metrics, and a crucial Two-Layer ROI Logic. This logic allows large, trusted signs to bypass the ROI check for stable tracking while forcing smaller, potential noise artifacts to remain within the defined Region of Interest.

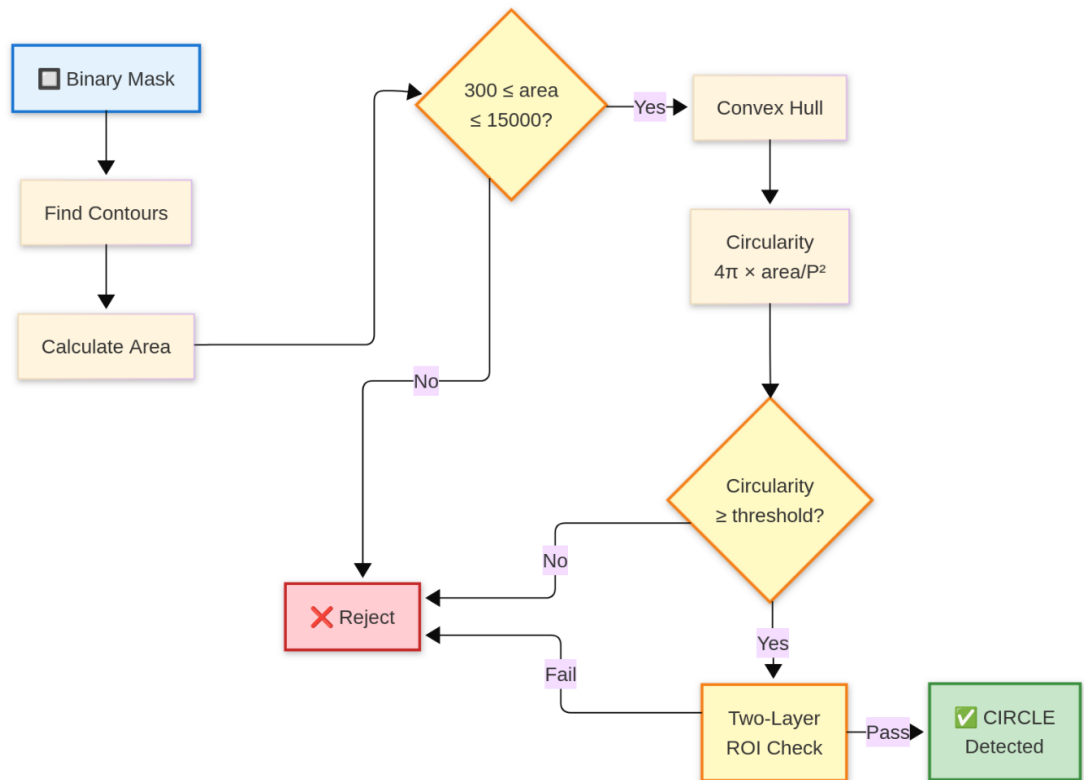


Figure 7: Circle Detection Pipeline

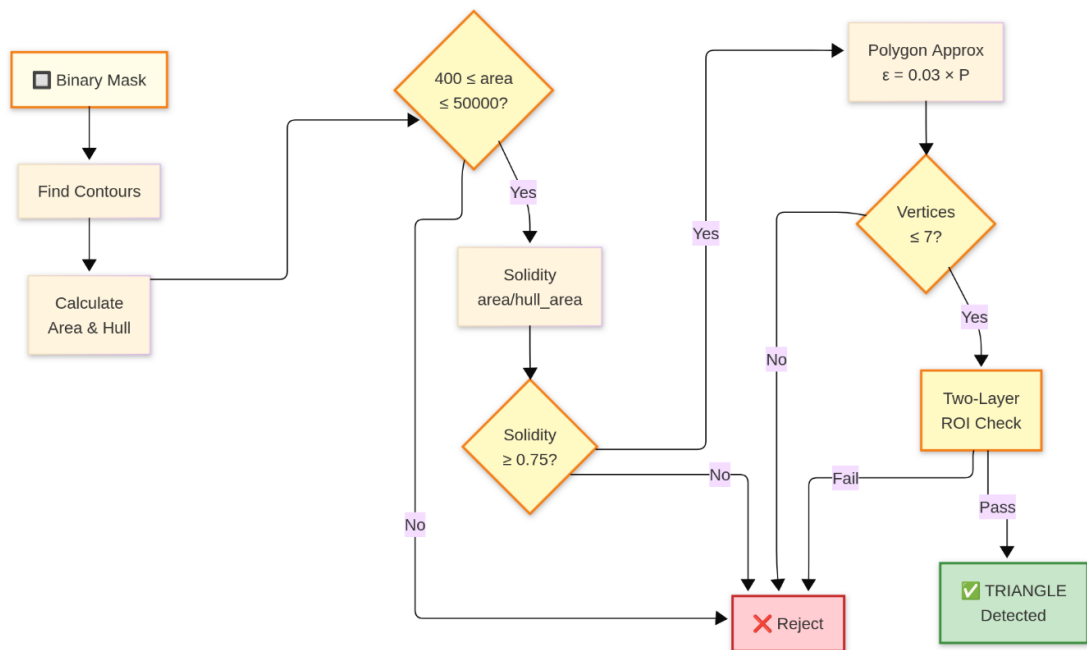


Figure 8: Triangle Detection Pipeline

```

def extract_circle_detections(mask, roi_params, color_type, min_area=300, max_area=15000):
    """Extract circular detections with two-layer ROI logic"""
    detections = []
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    for contour in contours:
        area = cv2.contourArea(contour)

        if area < min_area or area > max_area:
            continue

        hull = cv2.convexHull(contour)
        perimeter_hull = cv2.arcLength(hull, True)
        area_hull = cv2.contourArea(hull)
        x, y, w, h = cv2.boundingRect(hull)

        is_in_roi = is_bbox_in_roi((x, y, w, h), roi_params, overlap_threshold=0.5)

        # TWO-LAYER ROI LOGIC for circles:
        # Layer 1: Filter small noise outside ROI
        if area < CIRCLE_TRUST_THRESHOLD and not is_in_roi:
            continue # Reject small circles outside ROI (noise)
        # Layer 2: Trust large circles, bypass ROI check
        elif area >= CIRCLE_TRUST_THRESHOLD:
            pass # Allow large circles regardless of ROI (better tracking)
        # Layer 1: Allow small circles inside ROI
        elif area < CIRCLE_TRUST_THRESHOLD and is_in_roi:
            pass # Allow small circles inside ROI

        if perimeter_hull > 0:
            circularity = 4 * np.pi * (area_hull / (perimeter_hull * perimeter_hull))

            if area < CIRCLE_TRUST_THRESHOLD:
                circularity_param = CIRCLE_SMALL_CIRCULARITY
            else:
                circularity_param = CIRCLE_LARGE_CIRCULARITY

            if circularity > circularity_param:
                # Store bbox, color, and metrics
                metrics = {
                    'area': int(area_hull),
                    'circularity': round(circularity, 3),
                    'shape': 'circle'
                }
                detections.append((x, y, w, h), color_type, metrics))

    return detections

```

Figure 9: Circle Detection Function

```

def extract_triangle_detections(mask, roi_params, color_type, min_area=825, max_area=50000):
    """Extract triangular detections with two-layer ROI logic"""
    detections = []
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    for contour in contours:
        area = cv2.contourArea(contour)

        if area < min_area or area > max_area:
            continue

        x, y, w, h = cv2.boundingRect(contour)

        is_in_roi = is_bbox_in_roi((x, y, w, h), roi_params, overlap_threshold=0.5)

        # TWO-LAYER ROI LOGIC for triangles:
        # Layer 1: Filter small noise outside ROI
        if area < TRIANGLE_TRUST_THRESHOLD and not is_in_roi:
            continue # Reject small triangles outside ROI (noise)
        # Layer 2: Trust large triangles, bypass ROI check
        elif area >= TRIANGLE_TRUST_THRESHOLD:
            pass # Allow large triangles regardless of ROI (better tracking)
        # Layer 1: Allow small triangles inside ROI
        elif area < TRIANGLE_TRUST_THRESHOLD and is_in_roi:
            pass # Allow small triangles inside ROI

        hull = cv2.convexHull(contour)
        hull_area = cv2.contourArea(hull)

        if hull_area == 0:
            continue

        perimeter = cv2.arcLength(contour, True)
        if perimeter == 0:
            continue

        solidity = float(area) / hull_area

        if solidity <= TRIANGLE_MIN_SOLIDITY:
            continue

        # Approximate contour to polygon with epsilon proportional to perimeter
        epsilon = TRIANGLE_EPSILON_FACTOR * perimeter
        approx = cv2.approxPolyDP(contour, epsilon, True)
        num_vertices = len(approx)

        # Filter for triangular shapes (allow slight over-segmentation)
        if num_vertices <= TRIANGLE_MAX_VERTICES:
            # Store bbox, color, and metrics
            metrics = {
                'area': int(area),
                'solidity': round(solidity, 3),
                'shape': 'triangle'
            }
            detections.append(((x, y, w, h), color_type, metrics))

    return detections

```

Figure 10: Triangle Detection Function

Contour Approximation in Triangle Detection:

- The Douglas–Peucker approximation simplifies contours by removing redundant vertices.
- Setting $\varepsilon = 0.03 \times \textit{perimeter}$ balances shape fidelity and noise tolerance:
- smaller ε produces excessive vertices (sensitive to noise), whereas larger ε oversimplifies the contour and can distort triangles.

Contour Extraction and Initial Area Filtering:

This stage isolates potential traffic signs from the binary mask by defining acceptable size limits.

- `cv2.findContours`: Retrieves external contours (the boundaries of white blobs) from the cleaned binary mask.
- Area Calculation: `cv2.contourArea` computes the area of the current contour.
- Initial Filtering: The contour is rejected if its area falls outside the defined range `[min_area, max_area]`. This ensures small noise and excessively large, non-sign objects are discarded early.
- Area Range Selection:
 - These limits were derived empirically from the dataset.
 - Areas below the minimum usually correspond to small reflections or distant noise; areas above the maximum represent large red regions such as vehicles or background objects.

Shape Metrics – Convex Hull, Solidity, and Circularity:

- Convex Hull is computed to stabilize the contour against small notches or gaps on the sign boundary.
- Solidity: $\text{Solidity} = \text{Area} / \text{Hull Area}$ measures compactness: values near 1.0 indicate a solid shape, whereas lower values correspond to irregular noise.

- **Circularity:** $\text{Circularity} = 4\pi \times (\text{Area} / \text{Perimeter}^2)$ quantifies roundness; perfect circles yield ≈ 1.0 .

Two-Layer ROI Logic:

This is the core mechanism for spatial validation, combining area confidence with the Region of Interest (ROI) overlap.

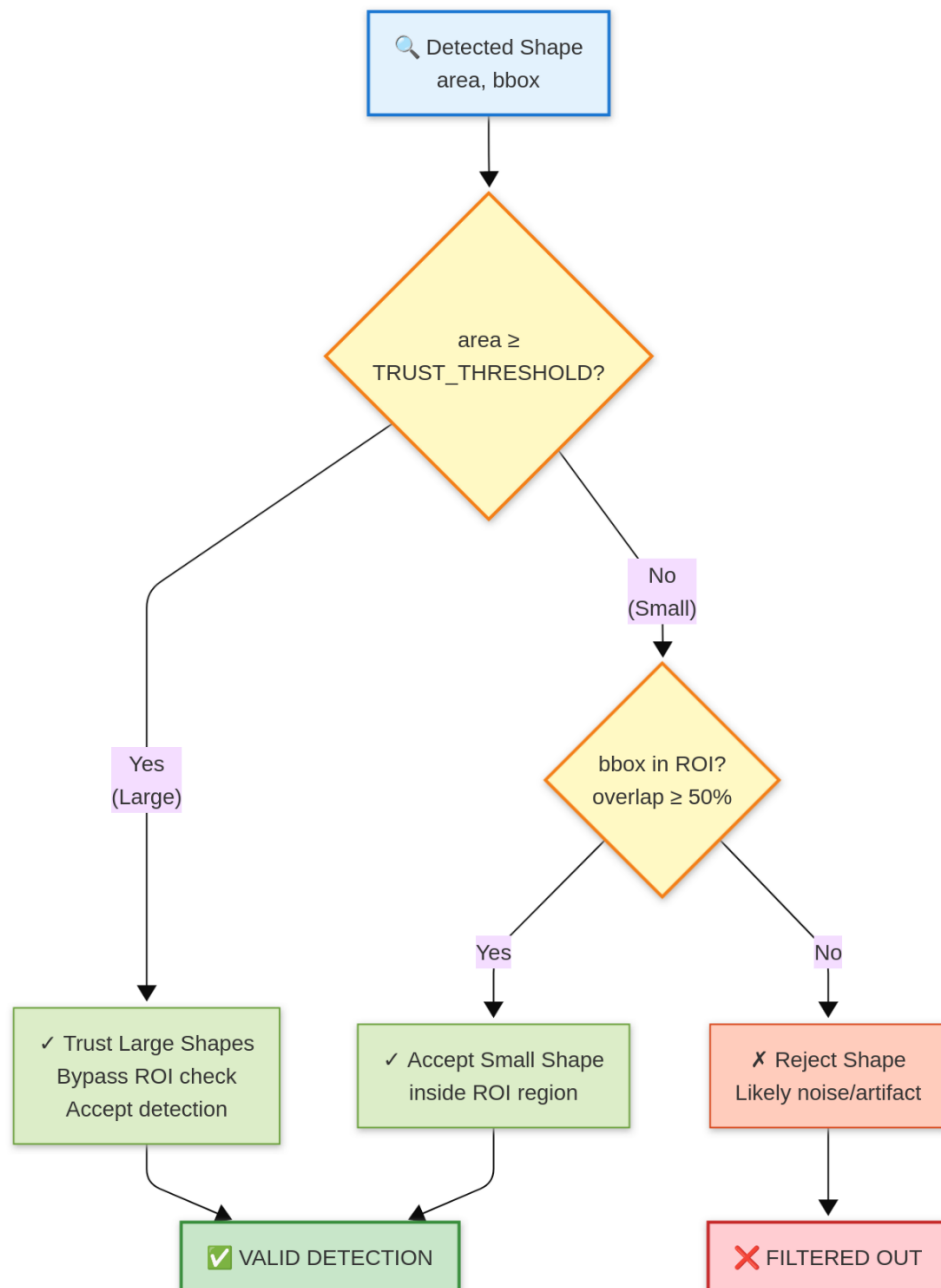


Figure 11: Two-Layer ROI Filtering diagram

```

# TWO-LAYER ROI LOGIC for circles:
# Layer 1: Filter small noise outside ROI
if area < CIRCLE_TRUST_THRESHOLD and not is_in_roi:
    continue # Reject small circles outside ROI (noise)
# Layer 2: Trust large circles, bypass ROI check
elif area >= CIRCLE_TRUST_THRESHOLD:
    pass # Allow large circles regardless of ROI (better tracking)
# Layer 1: Allow small circles inside ROI
elif area < CIRCLE_TRUST_THRESHOLD and is_in_roi:
    pass # Allow small circles inside ROI

```

Figure 12: Two-Layer ROI Logic for circles

```

# TWO-LAYER ROI LOGIC for triangles:
# Layer 1: Filter small noise outside ROI
if area < TRIANGLE_TRUST_THRESHOLD and not is_in_roi:
    continue # Reject small triangles outside ROI (noise)
# Layer 2: Trust large triangles, bypass ROI check
elif area >= TRIANGLE_TRUST_THRESHOLD:
    pass # Allow large triangles regardless of ROI (better tracking)
# Layer 1: Allow small triangles inside ROI
elif area < TRIANGLE_TRUST_THRESHOLD and is_in_roi:
    pass # Allow small triangles inside ROI

```

Figure 13: Two-Layer ROI Logic for triangles

```

def is_bbox_in_roi(bbox, roi_params, overlap_threshold=0.5):
    """Check if bbox is in ROI"""
    x, y, w, h = bbox
    roi_x_start, roi_y_start, roi_x_end, roi_y_end = roi_params

    x_max = x + w
    y_max = y + h

    inter_x1 = max(x, roi_x_start)
    inter_y1 = max(y, roi_y_start)
    inter_x2 = min(x_max, roi_x_end)
    inter_y2 = min(y_max, roi_y_end)

    if inter_x2 <= inter_x1 or inter_y2 <= inter_y1:
        return False

    inter_area = (inter_x2 - inter_x1) * (inter_y2 - inter_y1)
    bbox_area = w * h

    overlap_ratio = inter_area / bbox_area if bbox_area > 0 else 0

    return overlap_ratio >= overlap_threshold

```

Figure 14: is_bbox_in_roi function

- Bounding Box: `cv2.boundingRect` provides the rectangle for the `is_bbox_in_roi` custom function.
- ROI check:
 - Each color has its own predefined ROI in percentage of the full frame
 - These regions correspond to the areas where each sign type typically appears in the video.
 - A detection is accepted only if at least 50 % of its area overlaps with the ROI (`overlap_threshold = 0.5`).
 - This eliminates random color patches elsewhere in the frame.
- Trust Threshold: This value separates "trusted" signs from small, potentially noisy detections.
 - Small shapes ($\text{Area} < \text{Trust Threshold}$) are only accepted if `is_in_roi` is `True`, effectively filtering out noise far from the expected sign location.
 - Large shapes ($\text{Area} \geq \text{Trust Threshold}$) are accepted regardless of `is_in_roi`, preventing valid signs close to the camera from being lost due to minor bounding box shifts.

1.1.2.5 Step 5: Temporal Tracking

The Temporal Tracking step uses the `TemporalSignFilter` class to provide temporal consistency to the detected traffic signs. It works in two phases:

- Tracking (Association): It links detections across consecutive frames using IoU (Intersection over Union) and color-specific time gaps, building raw "tracks" for each sign.
- Validation and Smoothing: It filters out tracks that do not persist for a minimum duration to reject sporadic noise. Valid tracks are then post-processed using linear interpolation (to fill missing detections) and a moving average filter (to smooth the bounding box coordinates), resulting in stable, low-jitter visual output for the final video.


```

class TemporalSignFilter:
    """Filter traffic signs based on temporal consistency with color-specific parameters"""
    def __init__(self, fps, color_params=None):...
    def calculate_iou(self, box1, box2):...
    def add_detections(self, frame_num, detections):...
    def interpolate_missing_frames(self, track_data, color):...
    def smooth_bounding_boxes(self, track_data, window_size=5):...
    def build_detection_cache(self):...
    def get_validated_detections(self, frame_num):...
    def get_statistics(self):...

```

Figure 15: Temporal Filter Class

Tracking and Association:

This function is the core of the tracking system. It attempts to link a current detection to an existing track based on color consistency, proximity (IoU), and a maximum frame gap.

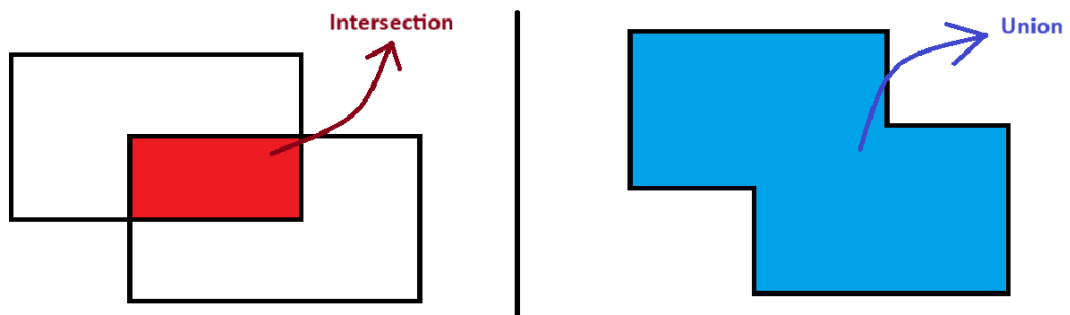


Figure 16: Intersection over Union

Intersection over Union (IoU) is a standard metric in computer vision, widely used in tasks like object detection and tracking. It quantifies the degree of overlap between two bounding boxes, providing a score that represents their spatial similarity.

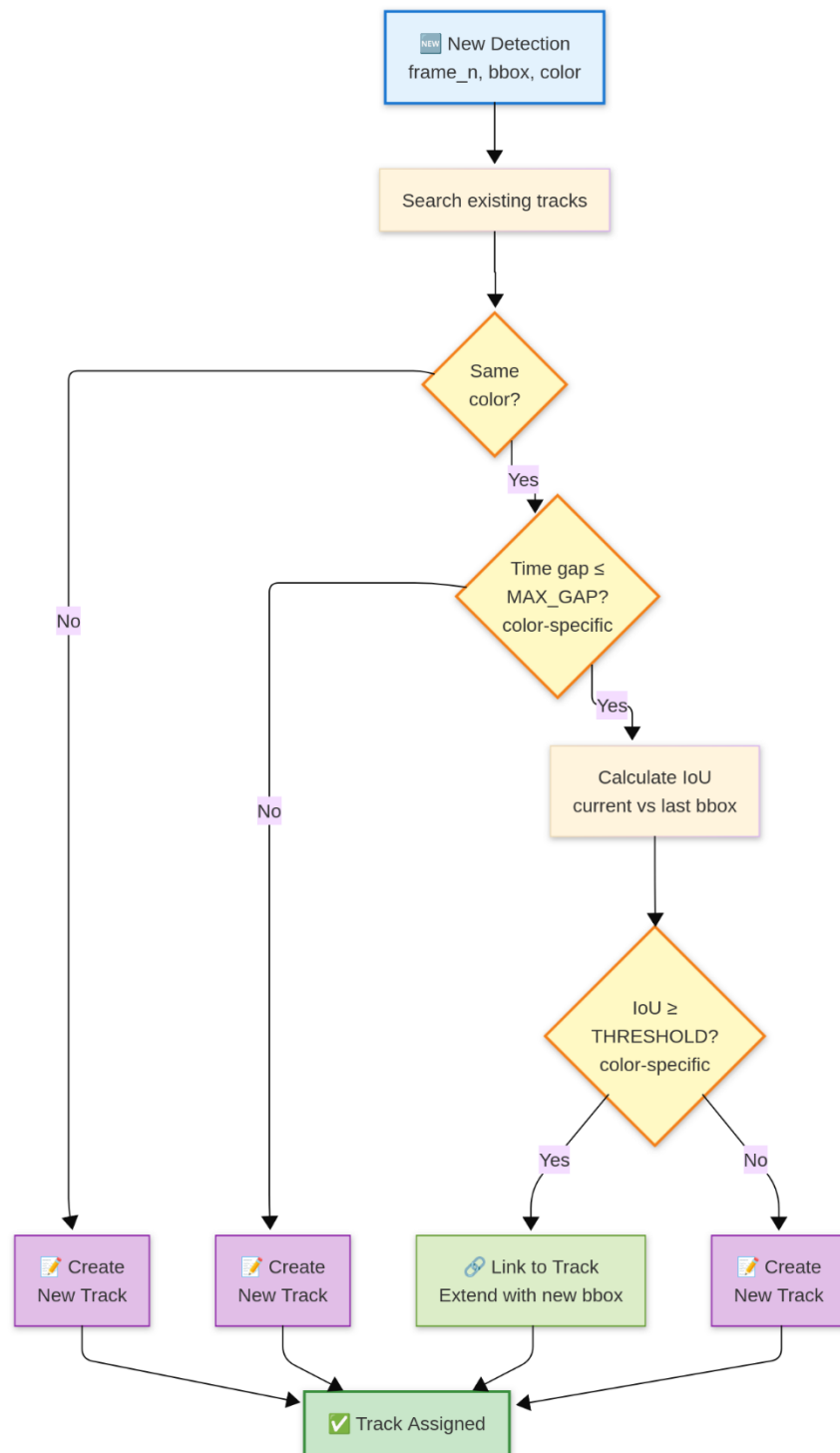


Figure 17: Temporal Tracking Pipeline

```

def add_detections(self, frame_num, detections):
    """Add detections from current frame with color-specific IoU threshold"""
    for detection_data in detections:
        bbox, color = detection_data[0], detection_data[1]
        # Extract metrics if available (for debugging)
        metrics = detection_data[2] if len(detection_data) > 2 else {}

        # Use color-specific IoU threshold
        iou_threshold = self.iou_thresholds.get(color, 0.3)
        max_gap = self.max_gap_frames.get(color, int(0.5 * self.fps))

        best_match_id = None
        best_iou = 0

        for track_id, track_data in self.tracks.items():
            if len(track_data) == 0:
                continue

            last_detection = track_data[-1]

            if (last_detection['color'] == color and
                frame_num - last_detection['frame'] <= max_gap):

                iou = self.calculate_iou(bbox, last_detection['bbox'])
                if iou > best_iou and iou >= iou_threshold:
                    best_iou = iou
                    best_match_id = track_id

        if best_match_id is not None:
            self.tracks[best_match_id].append({
                'frame': frame_num,
                'bbox': bbox,
                'color': color,
                'metrics': metrics
            })
        else:
            self.tracks[self.next_track_id] = [{
                'frame': frame_num,
                'bbox': bbox,
                'color': color,
                'metrics': metrics
            }]
            self.next_track_id += 1

```

Figure 18: add_detections function

- **Color and Time Gap Check:** A new detection is only considered for an existing track if it has the same color and occurs within the maximum frame gap (`max_gap`) of the track's last detection. This rejects unrelated objects and tracks that have disappeared for too long.
- **IoU Calculation:** The `calculate_iou` function determines the overlap between the new detection's bounding box and the last bounding box in the track.
- **Association Threshold:** If the IoU exceeds the color-specific `iou_threshold`, the detection is associated with that track.

1.1.2.6 Step 6: Track Validation

After all frames have been processed and tracks have been formed, this function applies to the final validation filter and prepares the data for rendering.

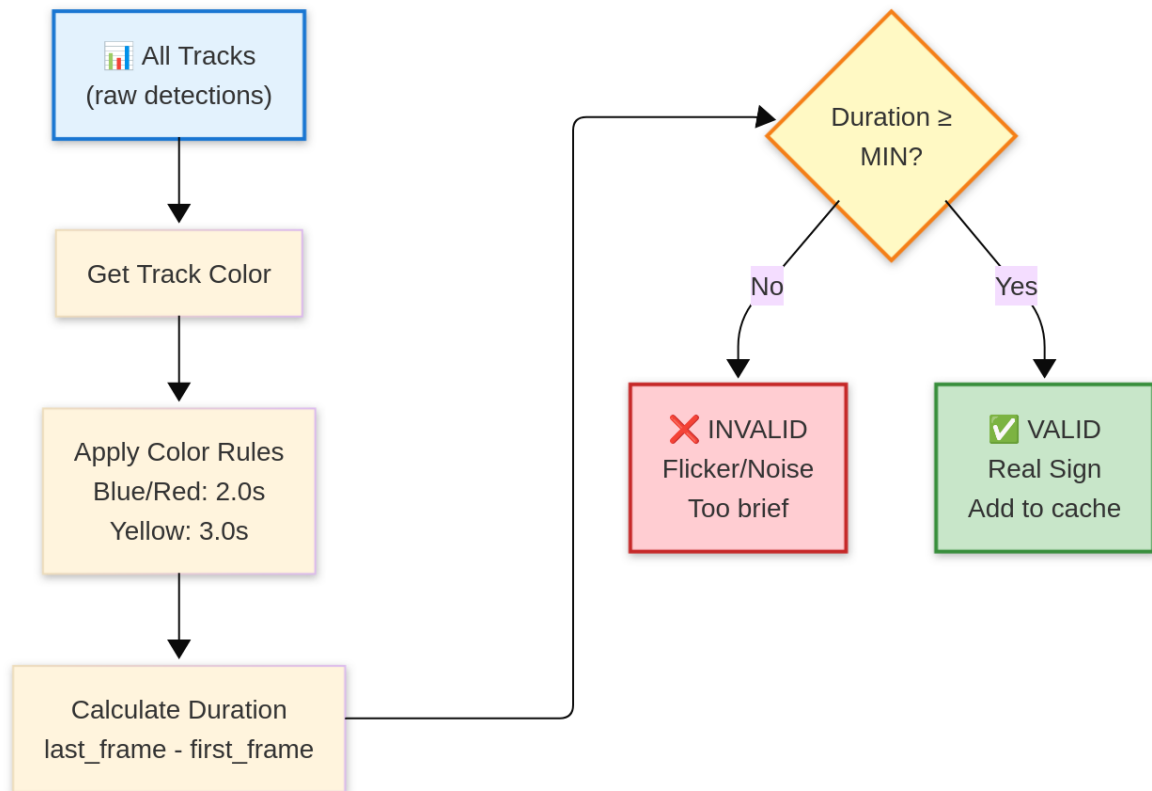


Figure 19: Track Validation Pipeline

```

def build_detection_cache(self):
    """
    Uses color-specific minimum duration for validation
    """
    print("    Building detection cache...")
    start = time.time()

    for track_id, track_data in self.tracks.items():
        if len(track_data) == 0:
            continue

        # Get color from first detection
        color = track_data[0]['color']

        # Use color-specific minimum frames
        min_frames_for_color = self.min_frames.get(color, int(2.0 * self.fps))

        first_frame = track_data[0]['frame']
        last_frame = track_data[-1]['frame']
        duration_frames = last_frame - first_frame + 1

        if duration_frames >= min_frames_for_color:
            interpolated = self.interpolate_missing_frames(track_data, color)
            smoothed = self.smooth_bounding_boxes(interpolated, window_size=5)

            for detection in smoothed:
                frame_num = detection['frame']
                if frame_num not in self._validated_cache:
                    self._validated_cache[frame_num] = []
                self._validated_cache[frame_num].append(
                    (detection['bbox'], detection['color'], detection.get('metrics', {}))
                )

    self._cache_built = True
    print(f"    Cache built in {time.time() - start:.2f}s ({len(self._validated_cache)} frames)")

```

Figure 20: build_detection_cache function

- **Minimum Duration Validation:** A track is only considered valid if its total span of frames (duration_frames) exceeds the color-specific min_frames threshold. This step eliminates fleeting, single-frame false positives or brief noise.
- **Post-Processing Pipeline:** Valid tracks undergo two final post-processing steps: Interpolation and Smoothing.

1.1.2.7 Step 7: Track Refinement

This step refines the validated tracks to ensure high stability and visual continuity for the final video output.

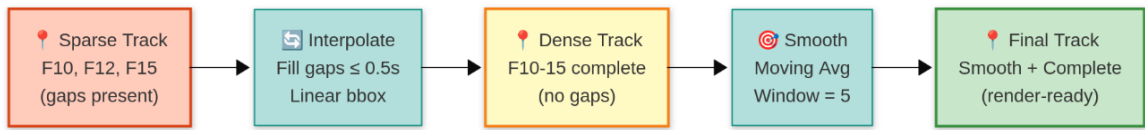


Figure 21: Interpolation and Smoothing Pipeline

Linear Interpolation:

This process fills coordinate gaps in the track using linear progression, ensuring a smooth path between two detected points.

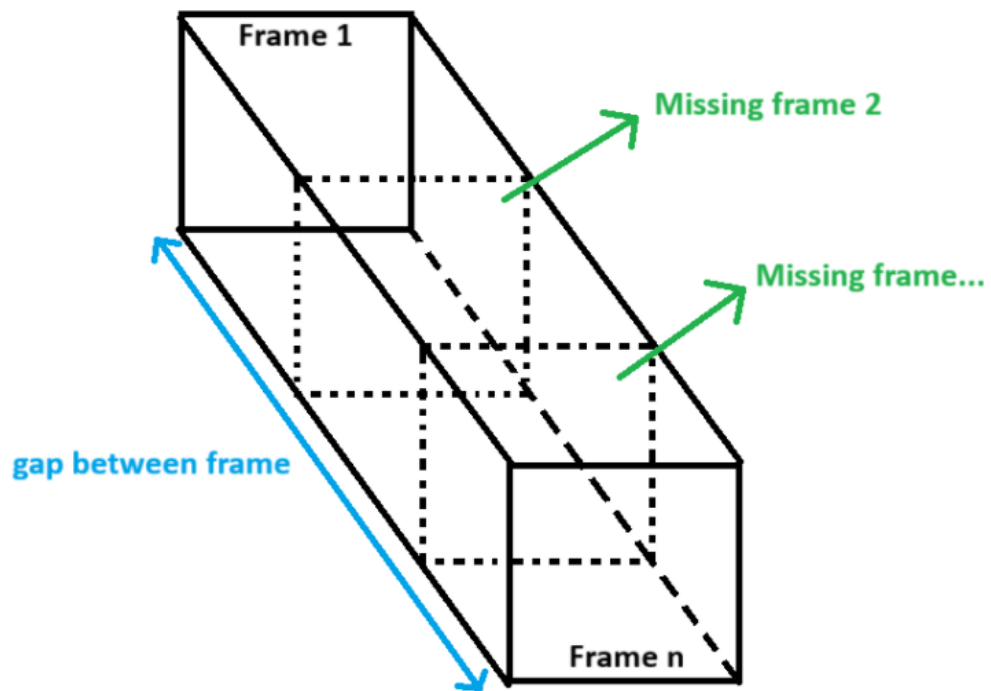


Figure 22: Linear Interpolation

Linear Interpolation is a mathematical method used to estimate a new value that lies between two known data points. In the context of computer graphics and object tracking, it is used to calculate the position of an object at an intermediate time step where no actual data (detection) exists.

```

def interpolate_missing_frames(self, track_data, color):
    """Fill missing frames with linear interpolation using color-specific max gap"""
    if len(track_data) < 2:
        return track_data

    # Get color-specific max gap
    max_gap = self.max_gap_frames.get(color, int(0.5 * self.fps))

    interpolated = []

    for i in range(len(track_data) - 1):
        current = track_data[i]
        next_det = track_data[i + 1]

        interpolated.append(current)

        frame_gap = next_det['frame'] - current['frame']

        if 1 < frame_gap <= max_gap:
            x1, y1, w1, h1 = current['bbox']
            x2, y2, w2, h2 = next_det['bbox']

            for j in range(1, frame_gap):
                alpha = j / frame_gap

                x_interp = int(x1 + (x2 - x1) * alpha)
                y_interp = int(y1 + (y2 - y1) * alpha)
                w_interp = int(w1 + (w2 - w1) * alpha)
                h_interp = int(h1 + (h2 - h1) * alpha)

                interpolated.append({
                    'frame': current['frame'] + j,
                    'bbox': (x_interp, y_interp, w_interp, h_interp),
                    'color': current['color'],
                    'metrics': current.get('metrics', {}),
                    'interpolated': True
                })

    interpolated.append(track_data[-1])
    return interpolated

```

Figure 23: interpolate_missing_frames function

- Mathematical Implementation:
 - For every frame i in the track, the new, smoothed coordinate P_{avg} is the arithmetic mean of the coordinate values within a fixed-size window W centered around i .
 - The window size is fixed at $W = 5$ (window_size = 5). The window spans from index start_idx to end_idx.
- where k is the frame index within the window W , and $|W|$ is the number of frames in the window (*which is* ≤ 5).

Smoothing Moving Average:

```

def smooth_bounding_boxes(self, track_data, window_size=5):
    """Smooth bounding boxes with moving average"""
    if len(track_data) < window_size:
        return track_data

    smoothed = []

    for i in range(len(track_data)):
        start_idx = max(0, i - window_size // 2)
        end_idx = min(len(track_data), i + window_size // 2 + 1)

        bboxes = [d['bbox'] for d in track_data[start_idx:end_idx]]

        x_avg = int(np.mean([b[0] for b in bboxes]))
        y_avg = int(np.mean([b[1] for b in bboxes]))
        w_avg = int(np.mean([b[2] for b in bboxes]))
        h_avg = int(np.mean([b[3] for b in bboxes]))

        smoothed_det = track_data[i].copy()
        smoothed_det['bbox'] = (x_avg, y_avg, w_avg, h_avg)
        smoothed.append(smoothed_det)

    return smoothed

```

Figure 24: smooth_bounding_boxes function

- Purpose: Reduces the high-frequency jitter and sudden jumps often seen in detection bounding boxes.
- Method: Applies a moving average filter to the bounding box coordinates. The box position in each frame is replaced by the average of its coordinates and the coordinates from the surrounding frames within the window. This results in a much smoother visual output.

1.1.3 Appendix for task 1

Performance Optimization by using Multi-Processing and Multi-Threading

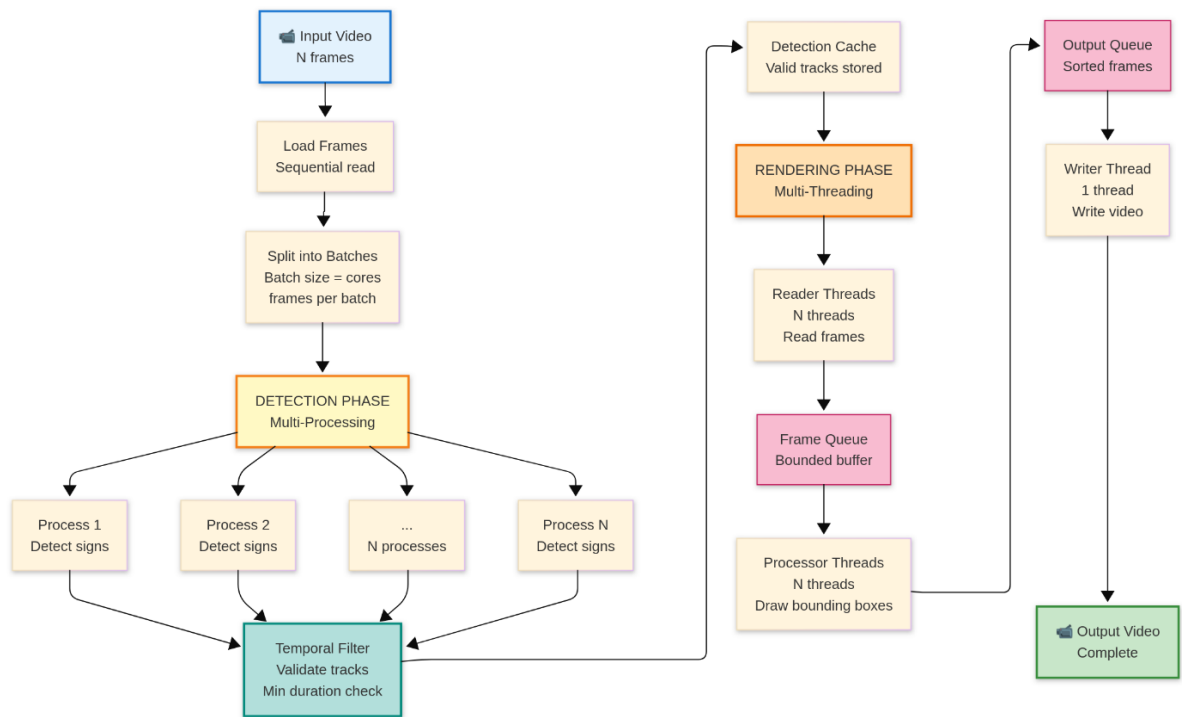


Figure 25: Multi-Processing and Multi-Threading diagram

Short description for parallel architecture:

1. Multi-Processing (Detection Phase)

- Splits video frames into batches for parallel processing
- Uses multiple parallel processes to detect traffic signs simultaneously
- Bypasses Python GIL for true parallelism on CPU-intensive OpenCV operations
- Number of processes automatically scales with available CPU cores

2. Multi-Threading (Rendering Phase)

- Uses reader threads to load video frames asynchronously
- Uses processor threads to draw bounding boxes in parallel
- Uses single writer thread to save output video sequentially
- Thread count automatically adapts to system resources

3. Dynamic Resource Allocation

- **Automatically detects CPU cores** and adjusts worker counts
- **Readers:** $\max(1, \min(4, \text{cores} \div 8))$

- **Workers:** $\max(1, \text{cores} - \text{readers} - 2)$
- **Buffer:** $\max(30, \min(200, \text{cores} \times 4))$ frames
- **Batch:** $\max(10, \min(50, \text{cores}))$ frames per batch
- Scales efficiently from low-end (4 cores) to high-end systems (32 + cores)

1.2 Task 2

1.2.1 Short description of main programming steps

Detailed Analysis of Each Region:

Diagram

1. Region 1 (Part 1 - Top): Normally noise.
2. Region 2 (Part 2 - Left): Horizontal lines stuck to digits, broken digit '4'.
3. Region 3 (Noise Part - Center-Bottom): Normally noise.
4. Region 4 (Part 3 - Right): Digits cut at right edge, digits '8' and '9' stuck together.

Diagram

Main Programming Steps

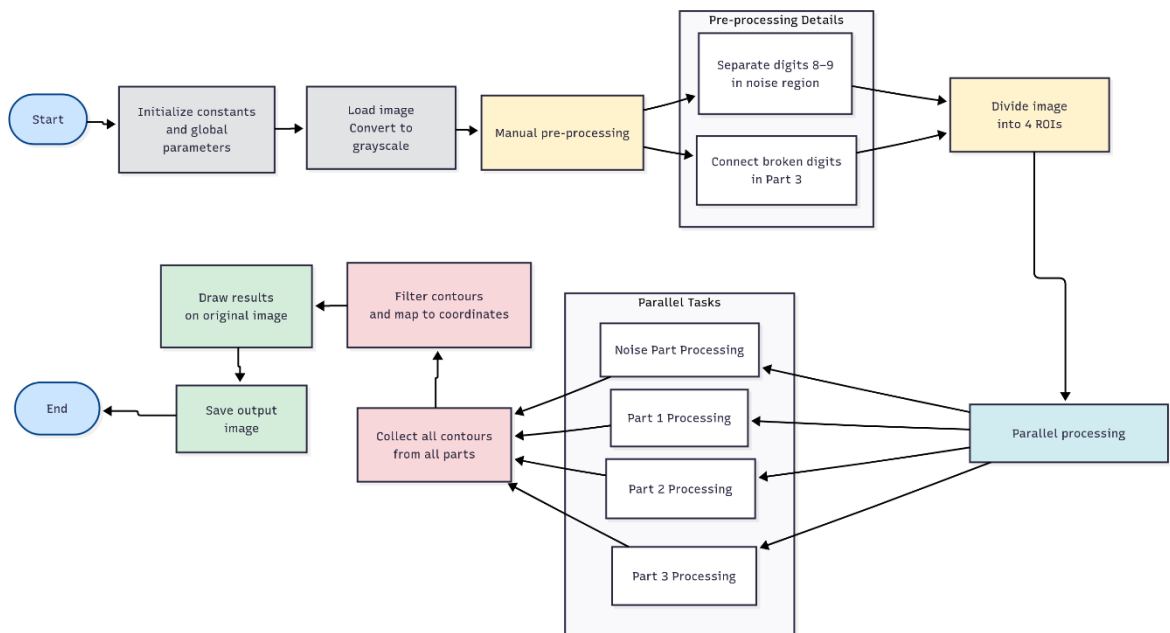


Figure 26: Overall Pipeline Architecture diagram

Detailed Step Descriptions:

1. Initialization & Constants Definition:

- Define threshold values for each region
- Define ROI coordinates for 4 regions
- Define filtering conditions (area, width, height ranges)

2. Image Loading & Conversion:

- Read an image color input.
- Convert to grayscale.

3. Manual Pre-processing:

- Separate digits 8-9: Draw white pixels at (330,530) with (height,width) = (10,10)
- Connect broken digits: Draw black pixels in Part 3 at (78,435) with (height,width) = (5,20)

4. Image Division:

- Extract 4 ROIs from grayscale image

5. Parallel Processing:

- Each region has its own specialized pipeline

6. Contours Collection:

- Gather contour lists from 4 pipelines

7. Filtering & Coordinate Mapping:

- Filter by area/geometry
- Transform local coordinates → global coordinates (offsets)

8. Drawing on Original Image:

- Draw green bounding boxes

9. Save Output:

- Save as Task2_output.png

1.2.2 Detailed Source Code Explanation

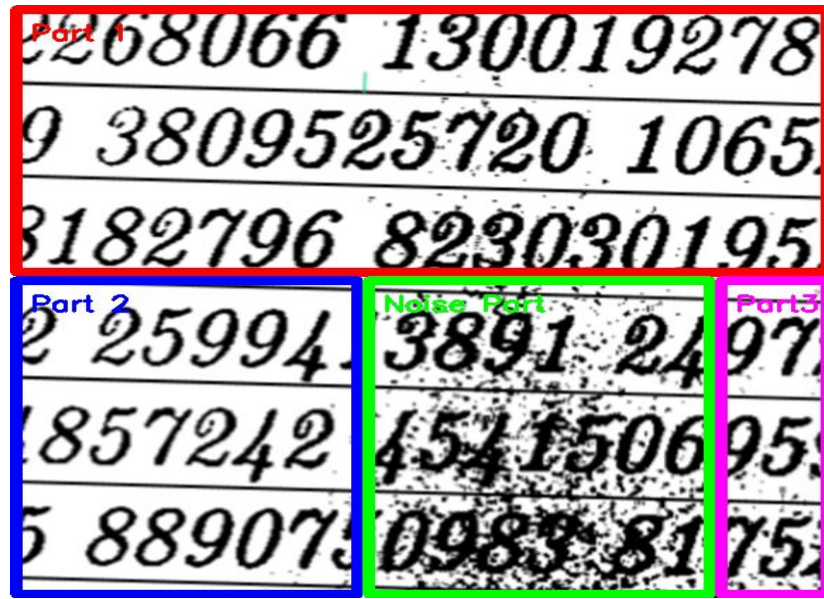


Figure 27: Input Image after Dividing into 4 regions

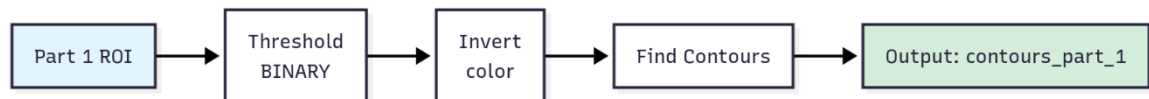
Region 1 Processing Pipeline (Part 1 - Clean Region):

Figure 28: Region 1 Pipeline Process diagram

```

'''II. Other Part Handling And Find Contours'''
gray_digits_img_2 = gray_digits_img.copy()

#Coordiante for ROI other parts
COORD_1 = 265
COORD_2 = 250
COORD_3 = 500
# Define Regions of Interest (ROI)
part_1 = gray_digits_img_2[:COORD_1, :] # Above the noise region
part_2 = gray_digits_img_2[COORD_2:, :COORD_1] # Left of the noise region
part_3 = gray_digits_img_2[:, COORD_3:] # Right of the noise region

#Connecting the half-cut digits on the right side of the image (For part_3 handling)
part_3[435:455, 78: 83] = 0

# --- LOGIC FOR PART 1 ---
# Binarization
_, bin_part_1 = cv2.threshold(part_1, THRESHOLD_PART_1, 255, cv2.THRESH_BINARY)
inv_bin_part_1 = cv2.bitwise_not(bin_part_1)

# Find Contours
contours_part_1, _ = cv2.findContours(inv_bin_part_1, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
color_part_1 = cv2.cvtColor(inv_bin_part_1, cv2.COLOR_GRAY2BGR)
for contour in contours_part_1:
    (x, y, w, h) = cv2.boundingRect(contour)
    cv2.rectangle(color_part_1, (x, y), (x + w, y + h), (0, 255, 0), 2)

```

Figure 29: Logical code for Part 1

1. `cv2.threshold(part_1, 160, 255, cv2.THRESH_BINARY):`

- **Input:** Grayscale Part 1
 - **Threshold:** 160 (very high for clean region)
 - **Logic:** if pixel > 160: pixel = 255 else: pixel = 0
 - **Output:** Black digits (0), white background (255)
2. cv2.bitwise_not(bin_part_1):
- **Reason:** cv2.findContours requires white objects (255), black background (0)
 - **Operation:** Invert → White digits (255), black background (0)
3. cv2.findContours(..., cv2.RETR_EXTERNAL,...):
- **RETR_EXTERNAL:** Only find outer contours, ignore nested contours
 - **CHAIN_APPROX_SIMPLE:** Compress contour, keep only endpoints of straight segments

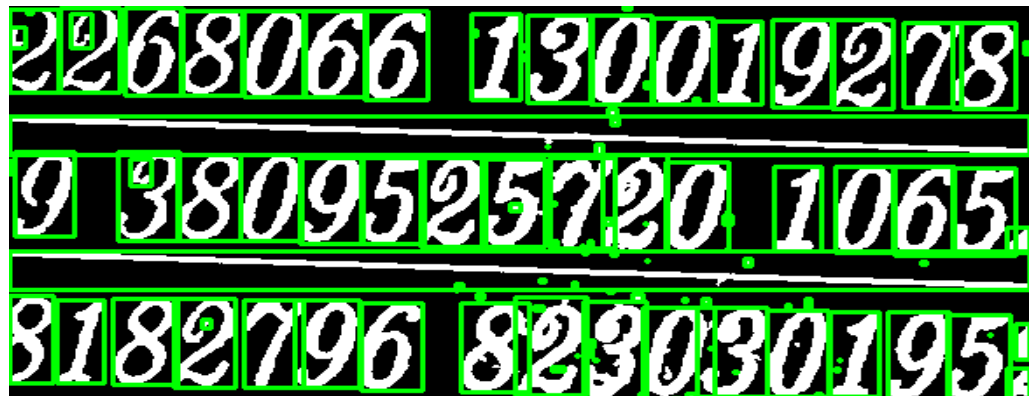


Figure 30: Part 1 ROI with Bounding Boxes

Noise Region Processing Pipeline (Noise Part - Complex Region):

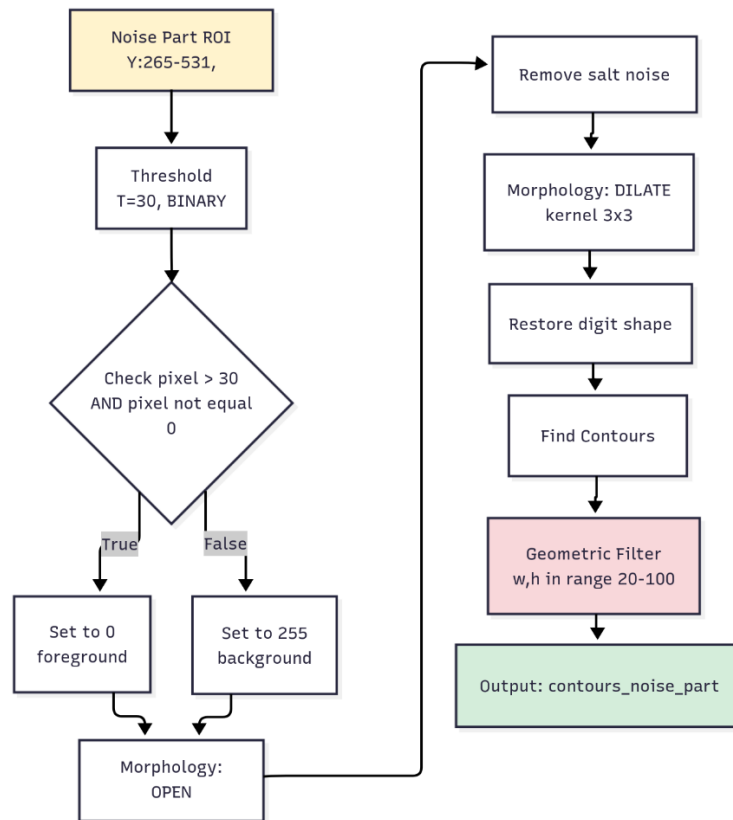


Figure 31: Noise Region Pipeline Process diagram

```

'''I. Noise Part Handling And Find Contours'''
# Small point has same color with background to separate digits 8 and 9 in the gray input image
gray_digits_img[530: 540, 330: 340] = 255

# Noise region coordinates
Y_START_NOISE = 265
Y_END_NOISE = gray_digits_img.shape[0] # Max image height
X_START_NOISE = 250
X_END_NOISE = 500
noise_part = gray_digits_img[Y_START_NOISE: Y_END_NOISE, X_START_NOISE: X_END_NOISE].copy()

# Binarization
condition = (noise_part > THRESHOLD_NOISE_PART) & (noise_part != 0)
noise_part[condition] = 0
noise_part[~condition] = 255

# Morphological Operations (Opening and Dilate)
opening_local_region = cv2.morphologyEx(noise_part, cv2.MORPH_OPEN, np.ones((3, 3), np.uint8))
noise_part_processed = cv2.dilate(opening_local_region, np.ones((3, 3), np.uint8))

# Find and filter contours
contours_noise_part_all, _ = utils.find_and_draw_contours(noise_part_processed)

# Filter contours by width and height to remove small noise
contours_noise_part = utils.filter_contours_by_coords(
    contours_noise_part_all,
    (0, gray_digits_img.shape[1]),
    (0, gray_digits_img.shape[0]),
    (NOISE_MIN_WH, NOISE_MAX_WH), #For width range
    (NOISE_MIN_WH, NOISE_MAX_WH) # For height range
)

```

Figure 32: Logical code for Noise Part

Step 1: Binarization with Special Condition

- More complex logic than regular threshold:
 - $\text{pixel} > 30$: Recover weak digit pixels
 - $\text{pixel} \neq 0$: Exclude completely black pixels (could be noise or background)



Figure 33: Binarization Noise Part ROI

Step 2: Morphological Opening (Noise Removal)

- **Effect:** Removes "salt noise" (tiny white dots)
- **Kernel 3×3 :** Large enough to remove noise, small enough not to destroy digit strokes

Step 3: Dilation (Shape Restoration)

- **Reason:** Opening "thins" digits, need dilation to restore
- **Effect:** Thickens strokes, connects lightly broken parts
- **Kernel 3×3 :** Sufficient for restoration without causing nearby digits to stick

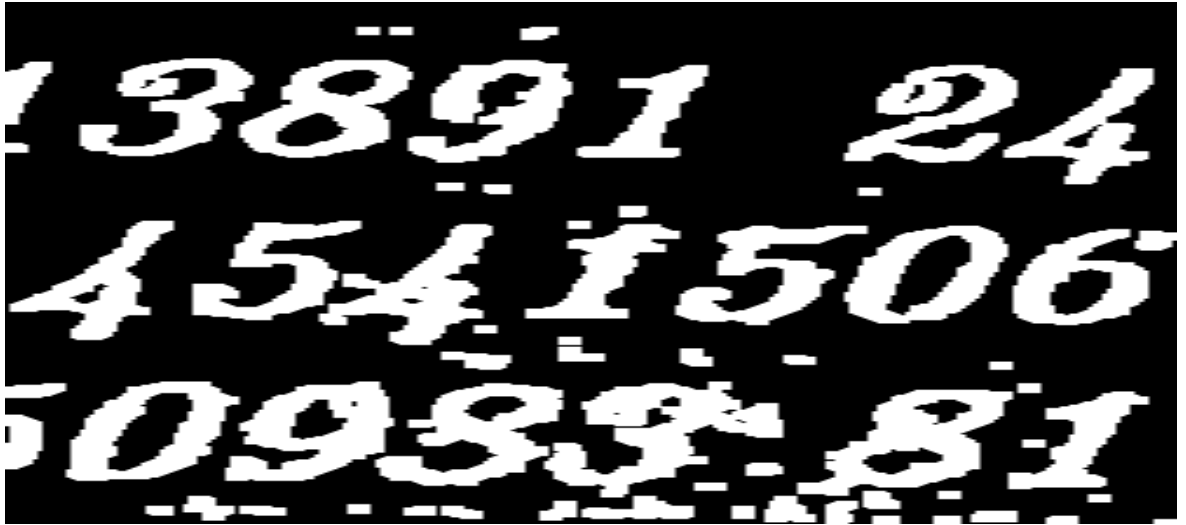


Figure 34: Processed Noise Part ROI

Step 4: Height and Width Filtering

- **Width/Height range** [NOISE_MIN_HW, NOISE_MAX_HW]: Based on observed actual digit sizes
- **Removes:**
 - Small noise ($w \text{ or } h < \text{NOISE_MIN_HW}$)
 - Abnormally large noise clusters ($w \text{ or } h > \text{NOISE_MAX_HW}$)
- **Retains:** Only contours with "digit-like" shapes



Figure 35: Noise Part ROI with Bounding Box

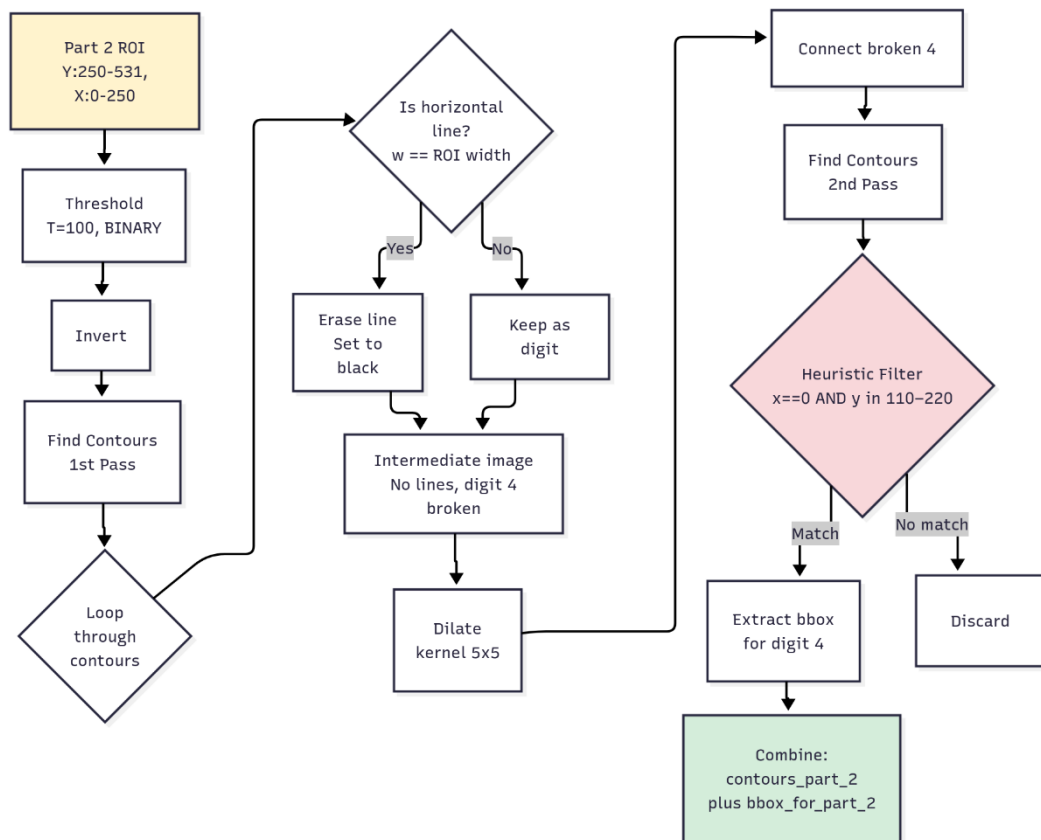
Region 2 Processing Pipeline (Part 2 - Lines Removal):

Figure 36: Region 2 Pipeline Process diagram

```

# --- LOGIC FOR PART 2 ---
# Binarization
_, bin_part_2 = cv2.threshold(part_2, THRESHOLD_PART_2, 255, cv2.THRESH_BINARY)
inv_bin_part_2 = cv2.bitwise_not(bin_part_2)

color_part_2 = cv2.cvtColor(inv_bin_part_2, cv2.COLOR_GRAY2BGR)

# Find initial contours and remove straight lines
contours_part_2, _ = cv2.findContours(inv_bin_part_2, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

for contour in contours_part_2:
    (x, y, w, h) = cv2.boundingRect(contour)
    # Check if contour spans the entire width (likely a horizontal line)
    if w == inv_bin_part_2.shape[1]:
        # Set the area corresponding to the line to black (0)
        color_part_2[y:y+h, x:x+w] = 0

# Apply Dilate and find new contours
part_2_gray = cv2.cvtColor(color_part_2, cv2.COLOR_BGR2GRAY)
inv_bin_part_with_dialte = cv2.dilate(part_2_gray, np.ones((5, 5), np.uint8))
contours_after_dialte, _ = cv2.findContours(inv_bin_part_with_dialte, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
color = cv2.cvtColor(inv_bin_part_with_dialte, cv2.COLOR_GRAY2BGR)
for contour in contours_after_dialte:
    (x, y, w, h) = cv2.boundingRect(contour)
    cv2.rectangle(color, (x, y), (x+w, y+h), (0, 255, 0), 2)

# Search for special bounding boxes (likely digit "4")
bbox_for_part_2 = None
for contour in contours_after_dialte:
    (x, y, w, h) = cv2.boundingRect(contour)
    # Look for digit "4"
    if x == 0 and (y > 110 and y < 220):
        bbox_for_part_2 = (x, y, w, h)

```

Figure 37: Logical code for Part 2

Step 1: Initial Binarization and Contour Finding

- **Action:** Binarize part_2 using its specific threshold and invert the colors. Find the contours (contours_part_2).
- **Purpose:** To get an initial set of bounding boxes for the digits.



Figure 38: Part 2 ROI with separated digit “4” in row 2

Step 2: Identify Problem

- **Action:** Observe the initial results and notice that the digit ‘4’ in the second row is not captured completely, while other digits are.
- **Purpose:** To identify a specific flaw in the initial processing.

Step 3: First Attempt at Correction (Dilation)

- **Action:** Apply a dilate technique with a (3, 3) kernel.
- **Purpose:** To attempt to fill in the missing parts of the digit “4”.



Figure 39: Part 2 ROI with digits “4” connect horizontal lines

Step 4: Identify New Problem from Dilation

- **Action:** Observe that the dilate operation causes the digits to connect with the nearby horizontal lines.
- **Purpose:** To recognize that simple dilation creates a new problem (connected components) that prevents correct contour finding.

Step 5: Remove Horizontal Lines

- **Action:** Iterate through the contours found in **Step 1** (contours_part_2). If a contour's width matches the full width of the **part_2 ROI**, identify it as a horizontal line and remove it by changing its color to the background color (black).

- **Purpose:** To create a "clean" image without the horizontal lines, so that dilation can be applied effectively.

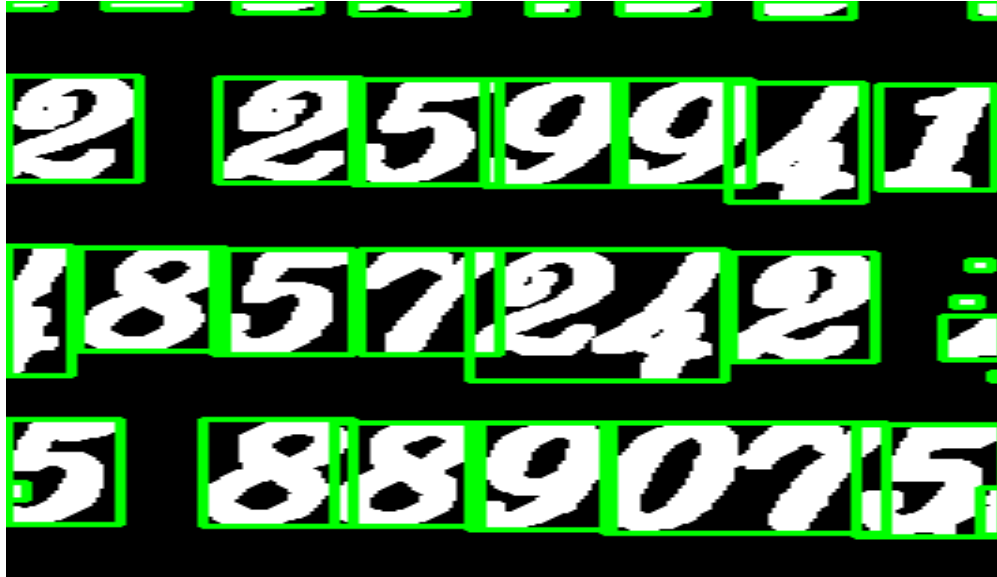


Figure 40: Part 2 ROI with wrong Bounding Box of digit “4”

Step 6: Second Attempt at Correction (Dilation on Clean Image)

- **Action:** Apply the dilate technique again, this time on the image from **Step 5** where the lines have been removed.
- **Purpose:** To successfully capture the complete shape of the digit “4” in the second row without it connecting to any lines.



Figure 41: Part 2 ROI with contours_part_2 and bbox_for_part_2

Step 7: Isolate and store the “4” Digit's Bounding Box

- **Action:** Find the contours on the image from **Step 6** (contours_after_dilate). Specifically identify the bounding box (coordinates, width, height) for the newly completed digit “4” and save it into a separate tuple (bbox_for_part_2).
- **Purpose:** To only add the missing digit “4” to the results. The result for part_2 is a combination of the original good contours from Step 1 (contours_part_2) and the newly found bounding box for the digit “4” (bbox_for_part_2)

Region 3 Processing Pipeline (Part 3 - Cut Digits):

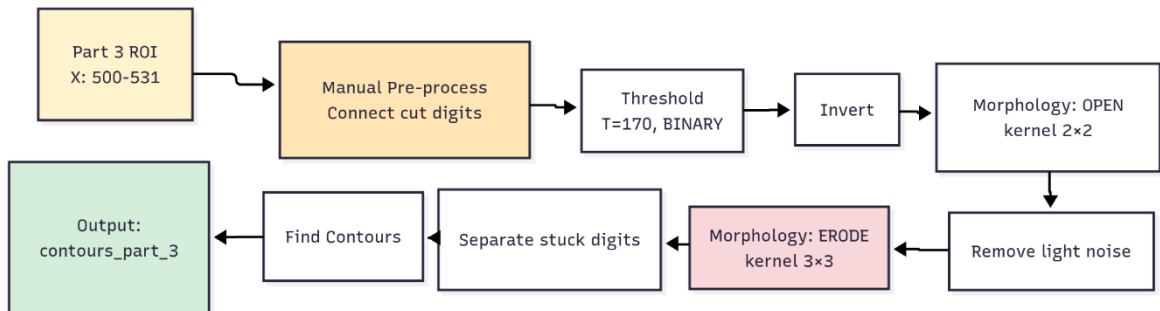


Figure 42: Region 3 Pipeline Process diagram

```

# --- LOGIC FOR PART 3 ---
# Binarization
_, bin_part_3 = cv2.threshold(part_3, THRESHOLD_PART_3, 255, cv2.THRESH_BINARY)
inv_bin_part_3 = cv2.bitwise_not(bin_part_3)

# Morphological Operations (Opening and Erode)
inv_bin_part_3 = cv2.morphologyEx(inv_bin_part_3, cv2.MORPH_OPEN, np.ones((2, 2), np.uint8))
inv_bin_part_3 = cv2.erode(inv_bin_part_3, np.ones((3, 3), np.uint8))

# Find Contours
contours_part_3, _ = cv2.findContours(inv_bin_part_3, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
new_color_part_3 = cv2.cvtColor(inv_bin_part_3, cv2.COLOR_GRAY2BGR)
  
```

Figure 43: Logical code for Part 3

Erosion Technique:

- **Effect:** "Shrinks" all foreground pixels inward
- **Result:**
 - Digits shrink → gap between them increases
 - Lightly stuck points are cut

- Creates clear boundaries between digits

Opening before Erode:

- **Purpose:** Clean small "salt" noise before erosion

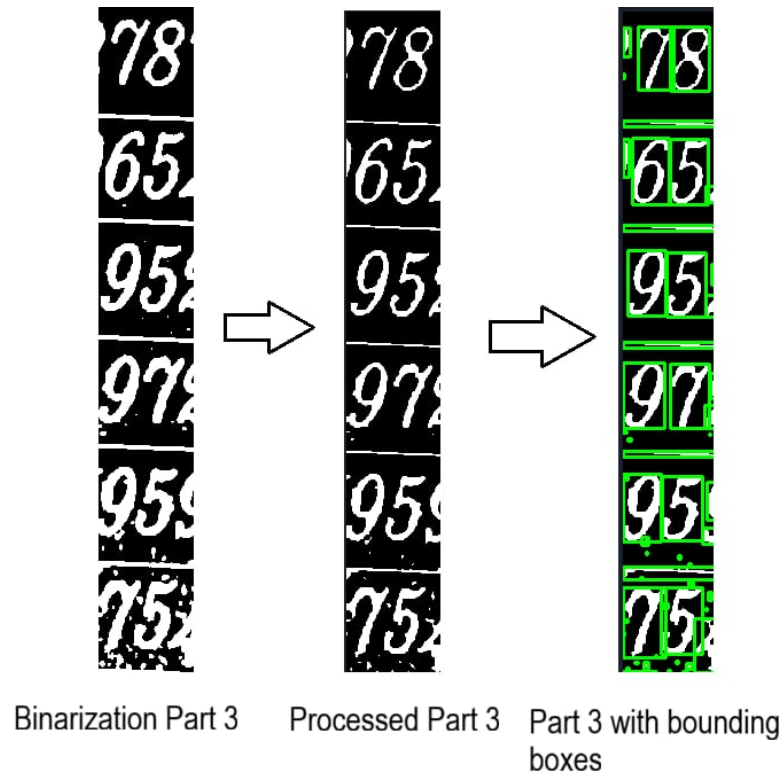


Figure 44: The Processing Pipeline Image of Part 3 ROI

Recombination and Final Filtering:

The offset parameters for applying contours onto the original image. Additionally, there are the area conditions to filter noise for part 1, part 2, and part 3.

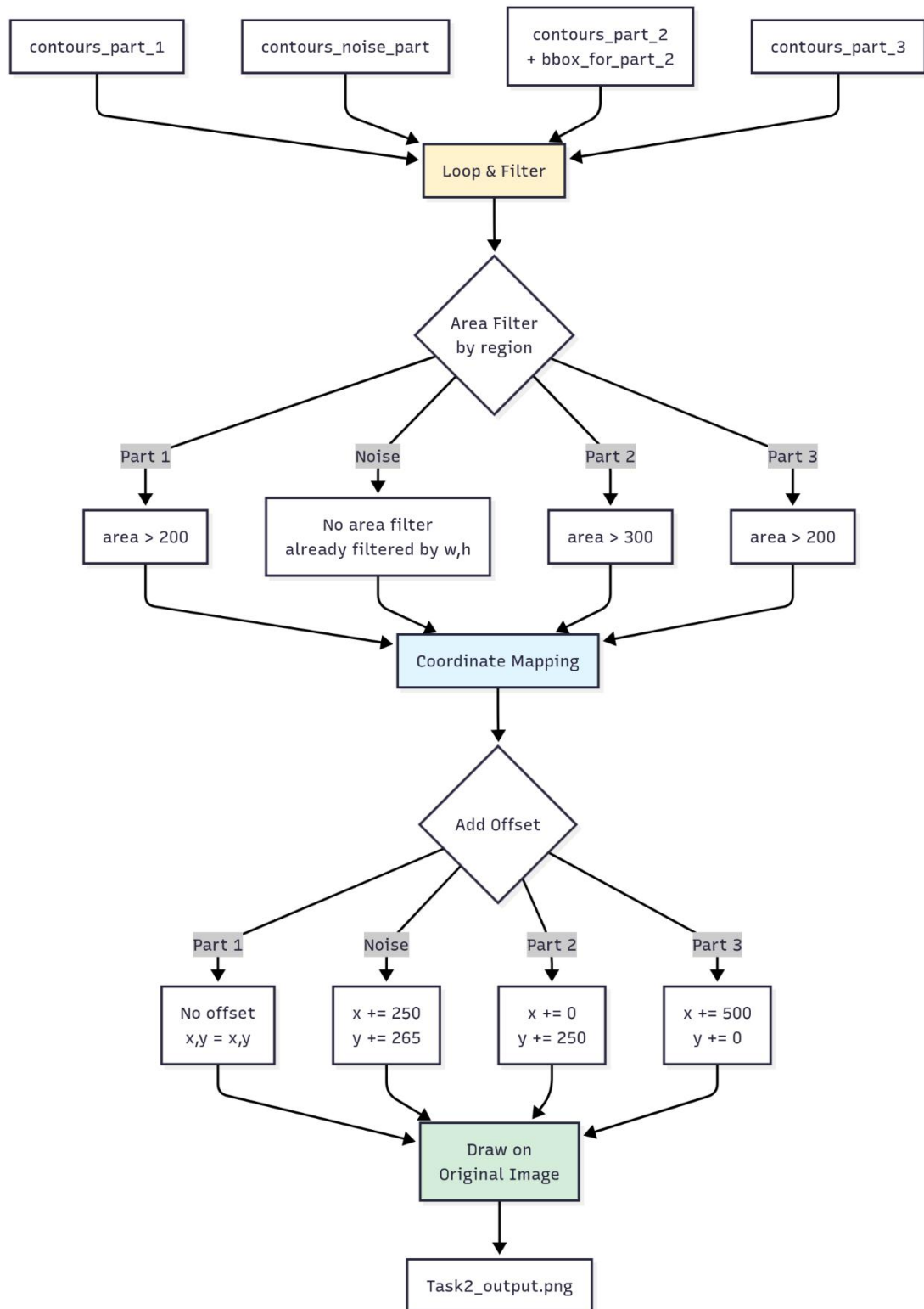


Figure 45: Recombination and Final Filtering Process Flowchart diagram

```
'''IV. Apply Contours to Original Image'''

# Offset values to map ROI coordinates back to the original image
OFFSET_X_NOISE_PART = X_START_NOISE # 250
OFFSET_Y_NOISE_PART = Y_START_NOISE # 265
OFFSET_X_PART2 = 0
OFFSET_Y_PART2 = COORD_2 # 250
OFFSET_X_PART3 = COORD_3 # 500
OFFSET_Y_PART3 = 0

# Area conditions
AREA_MIN_PART_1 = 200
AREA_MAX_PART_1 = 1500

AREA_MIN_PART_2 = 300
AREA_MAX_PART_2 = 1500

AREA_MIN_PART_3 = 200
AREA_MAX_PART_3 = 1500
```

Figure 46: Offsets and Area Conditions

Applying the contours of each part onto the input image. Filtering noise based on the defined area conditions.

```
for contour in contours_part_1:
    if cv2.contourArea(contour) > AREA_MIN_PART_1 and cv2.contourArea(contour) < AREA_MAX_PART_1:
        x, y, w, h = cv2.boundingRect(contour)
        # No offset needed for Part 1 (starts at 0,0)
        cv2.rectangle(digits_img, (x, y), (x + w, y + h), (0, 255, 0), 2)
# Noise Part: Draw bounding boxes with offset
for contour in contours_noise_part:
    x, y, w, h = cv2.boundingRect(contour)
    x = x + OFFSET_X_NOISE_PART
    y = y + OFFSET_Y_NOISE_PART
    cv2.rectangle(digits_img, (x, y), (x + w, y + h), (0, 255, 0), 2)
# Part 2: Draw bounding boxes with offset
for contour in contours_part_2:
    area = cv2.contourArea(contour)
    if area > AREA_MIN_PART_2 and area < AREA_MAX_PART_2:
        x, y, w, h = cv2.boundingRect(contour)
        # Remove bounding boxes of horizontal lines (already filtered in logic, but safe to check again)
        if w == part_2.shape[1]:
            continue
        x = x + OFFSET_X_PART2
        y = y + OFFSET_Y_PART2
        cv2.rectangle(digits_img, (x, y), (x + w, y + h), (0, 255, 0), 2)
# Applying for digit "4" in part 2 (bbox_for_part_2)
(x, y, w, h) = bbox_for_part_2
x = x + OFFSET_X_PART2
y = y + OFFSET_Y_PART2
cv2.rectangle(digits_img, (x, y), (x + w, y + h), (0, 255, 0), 2)
# Part 3: Draw bounding boxes with offset
for contour in contours_part_3:
    if cv2.contourArea(contour) > AREA_MIN_PART_3 and cv2.contourArea(contour) < AREA_MAX_PART_3:
        x, y, w, h = cv2.boundingRect(contour)
        if w == part_3.shape[1]:
            continue
        x = x + OFFSET_X_PART3
        y = y + OFFSET_Y_PART3
        cv2.rectangle(digits_img, (x, y), (x + w, y + h), (0, 255, 0), 2)
```

Figure 47: Applying Contours for Input Image

CHAPTER 2. Task results

2.1 Task 1

2.1.1 Output Image for Task 1

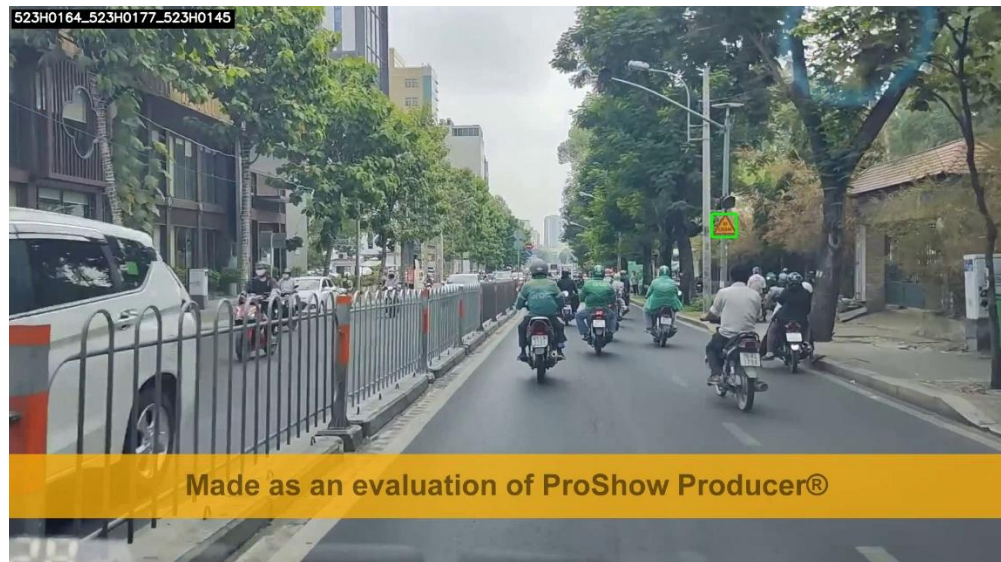


Figure 48: Output Frame 1

This frame demonstrates the successful automatic detection of a single traffic sign (Warning sign) in the road environment. The sign is accurately localized and enclosed within a green bounding box, confirming the algorithm's functionality.

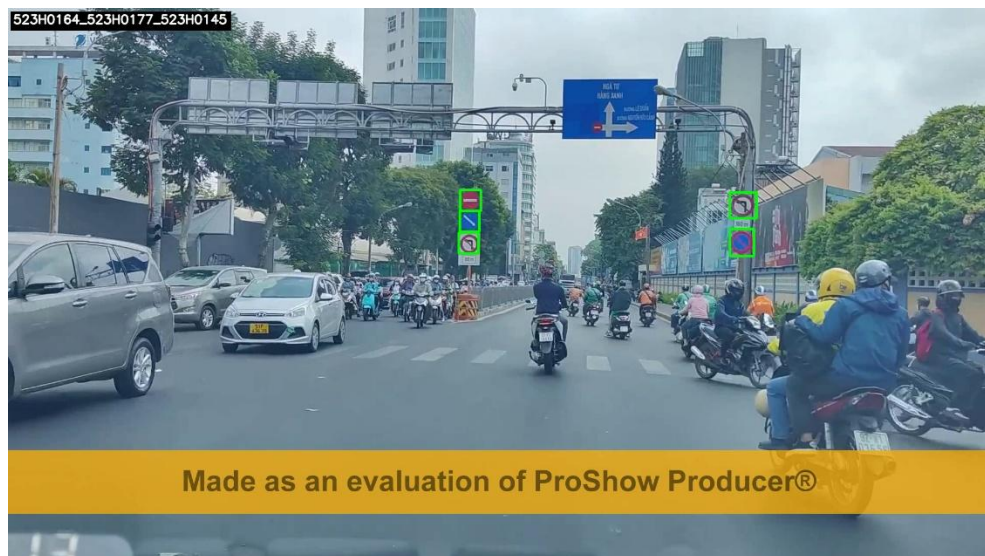


Figure 49: Output Frame 2

This image shows the algorithm's capability to simultaneously detect multiple traffic signs (Prohibition and Mandatory signs) under complex urban conditions. Each detected sign is correctly marked by a distinct green rectangle.



Figure 50: Output Frame 3

The frame confirms the successful tracking and detection of two traffic signs, one of which is at a smaller scale/greater distance. All detected signs are correctly surrounded by green rectangular bounding boxes as required.

2.1.2 Output Video for Task 1

<https://drive.google.com/file/d/1PyvBGbG7hGu0n0OGBICpR0U8qqJmV1K-/view>

2.2 Output Task 2

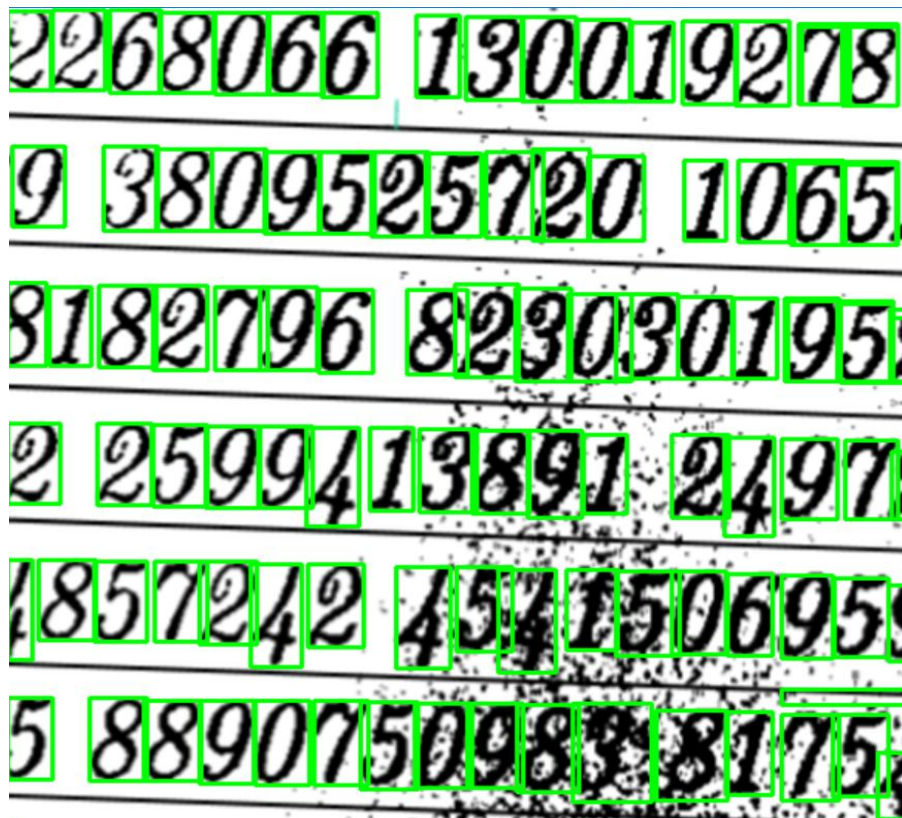


Figure 51 Final Result of task 2

This image shows the final output for Task 2, where the objective was to detect each digit in the input image. The image processing algorithm successfully localized and enclosed every digit with an accurate green rectangular bounding box.